ECE551: Digital System Design and Synthesis

Fall 2026

Coding Style & Synthesis Implications

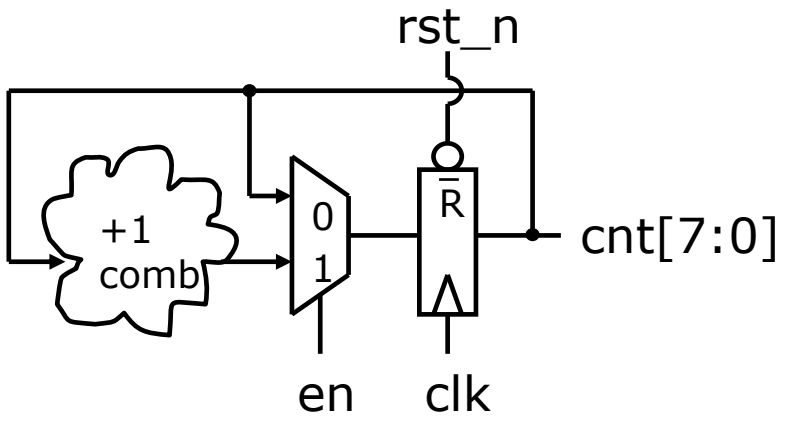
Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Behavioral Coding Style

- What have we learned?
  1. Sequential elements (flops & latches) should be inferred using non-blocking (<=)
  2. Combinational logic should be inferred using blocking (=)
  3. Blocking and non-blocking should not be mixed in the same **always** block
  4. Plus 5 other guidelines of good coding outlined in the Cummings SNUG paper
- Following these guidelines strictly will give us pretty verbose code…
- We can blur the boundaries a bit, now that we know what's happening
  - For more concise & readable code

Look at this example:

(8-bit counter with reset & enable)



Increment logic & mux are combinational ➔ blocking

Flop is sequential. ➔ non-blocking

Behavioral Coding Style

- Following all the rules…

```
module rule_cnt(clk,rst_n,en,cnt);

input clk,rst_n;
output reg [7:0] cnt;

reg [7:0] nxt_cnt;

always_ff @(posedge clk, negedge rst_n)
   if (!rst_n)
      cnt <= 8'h00;
   else
      cnt <= nxt_cnt;

always_comb
  if (en)
      nxt_cnt = cnt + 1;
  else
      nxt_cnt = cnt;

endmodule
```

Nothing wrong with this code.
Just a little verbose.

Use dataflow?

```
assign nxt_cnt = (en) ? cnt+1 : cnt;
```

More concise and readable code

```
module better_cnt(clk,rst_n,en,cnt);

input clk,rst_n;
output reg [7:0] cnt;

always_ff @(posedge clk, negedge rst_n)
   if (!rst_n)
      cnt <= 8'h00;
   else if (en)
      cnt <= cnt + 1;

endmodule
```

Code infers combinational using non-blocking assignment

Timer Full Checking

- Counters are commonly used as timers to time/delay some amount of time.
- Often your “end” condition is when the timer is full.
- Having variable timer width could be handy for experimenting with delay length

- If you initialize to 0 and check for timer full by using a reduction & then the rest of the code is “width agnostic”.

```
logic [WIDTH-1:0] tmr;

always_ff @(posedge clk)
   if (clr_tmr)
      tmr <= 0;
   else
      tmr <= tmr + 1;

assign time_over = &tmr;
```

Presetting a Counter

- The previous example of a parametrized WIDTH worked nicely because the register was initialized to 0.
- What if you have an application where you need a register or counter preset to all 1's? (perhaps a down counter).

- System Verilog offers a new feature called a vector fill token ' (e.g., '1, '0, 'x, 'z)

```
logic [WIDTH-1:0] cnt;

always_ff @(posedge clk)
   if (set_cnt)
     cnt <= '1;
   else
      cnt <= cnt - 1;

assign not_done = |cnt;
```

Ring Counter

- What do you think of this…

```
module ring_counter (count, enable, clock, reset);
  output reg     [7:0]    count;
  input          enable, reset, clock;

  always @ (posedge clock or posedge reset)
    if (reset == 1'b1)   count <= 8'b0000_0001;
    else if (enable == 1’b1) begin
        case (count)
           8’b0000_0001: count <= 8’b0000_0010;
           8’b0000_0010: count <= 8’b0000_0100;
           …
           8’b1000_0000: count <= 8’b0000_0001;
           default: count <= 8’bxxxx_xxxx;
        endcase
    end
endmodule
```

Better code

```
module ring_counter (count, enable, clock, reset_n);
  output reg     [7:0]    count;
  input          enable, reset_n, clock;

  always_ff @  (posedge clock or negedge reset_n)
    if (!reset_n)
      count <= 8'b0000_0001;
    else if (enable)
      count <= {count[6:0], count[7]};
endmodule
```

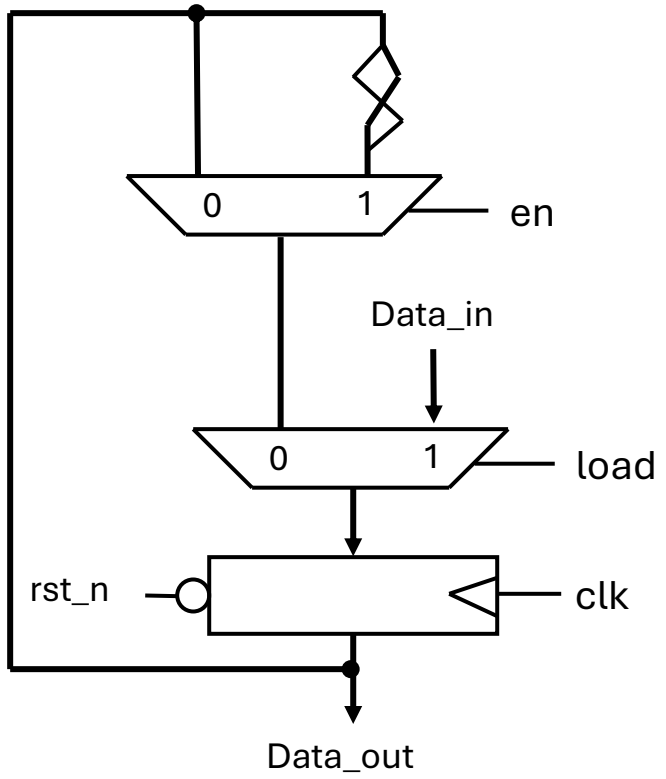
- Use vector concatenation in this example to be more explicit about desired behavior/implementation
  - More concise
  - Does not rely on synthesis tool to be smart and reduce your logic for you.

Rotator

- Think what this code implies … How will it synthesize?
- What would this block be used for?

```
module rotator (Data_out, Data_in, load, clk, rst_n);
  output reg     [7:0]   Data_out;
  input [7:0]    Data_in;
  input          load, clk, rst_n;

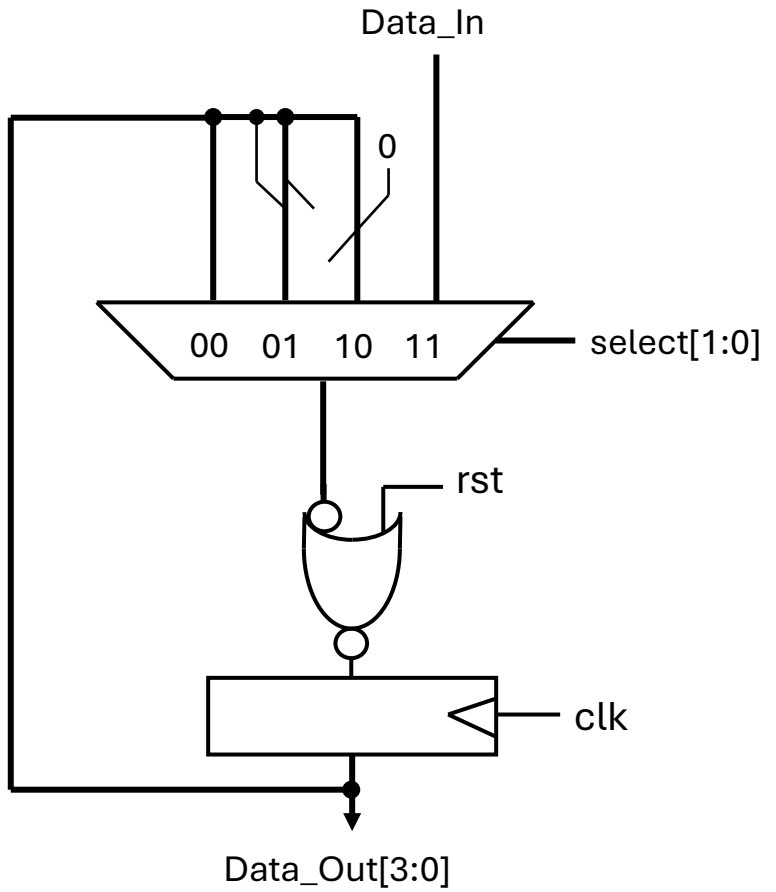
always_ff @  (posedge clk or negedge rst_n)
     if (!rst_n)          Data_out <= 8'b0;
     else if (load)       Data_out <= Data_in;
     else if (en)         Data_out <= {Data_out[6:0], Data_out[7]};
endmodule
```



Shifter

- Think what this code implies … How will it synthesize?
- Is the reset synchronous or asynchronous?
- Why was the MSB replicated on the ÷ by 2?

```
always_ff @ (posedge clk) begin
  if (rst) Data_Out <= 0;
  else  case (select[1:0])
    2'b00:  Data_Out <= Data_Out;                    // Hold
    2'b01:  Data_Out <= {Data_Out[3], Data_Out[3:1]}; // ÷ by 2
    2'b10:  Data_Out <= {Data_Out[2:0], 1'b0};        // X by 2
    2'b11:  Data_Out <= Data_In;                      // Parallel Load
    endcase
  end
endmodule
```



if…else if…else statement

- General form:

```
if (condition) begin
   <statement1>;
   <statement2>;
end
```

```
if (condition)
  begin
    <statement1>;
    <statement2>;
  end
else
  begin
    <statement3>;
    <statement4>;
  end
```

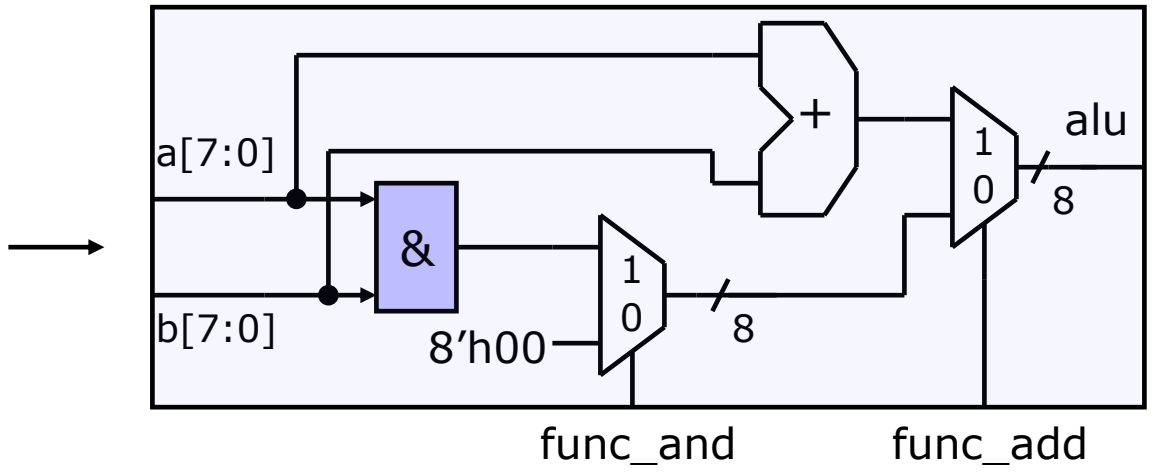
```
if (condition)
  begin
    <statement1>;
    <statement2>;
  end
else if (condition2)
  begin
    <statement3>;
    <statement4>;
  end
else
  begin
    <statement5>;
    <statement6>;
  end
```

- Multiple else if's can be strung along indefinitely

if…else if…else statement

- How does an if…else if…else statement synthesize?
  - Does not conditionally "execute" block of "code"
  - Does not conditionally create hardware
  - It makes a multiplexer or selecting logic
- Generally:
  - Hardware for both paths are created
  - Both paths compute simultaneously
  - The result is selected depending on the condition

```
if (func_add)
    alu = a + b;
else if (func_and)
    alu = a & b;
else
    alu = 8'h00;
```

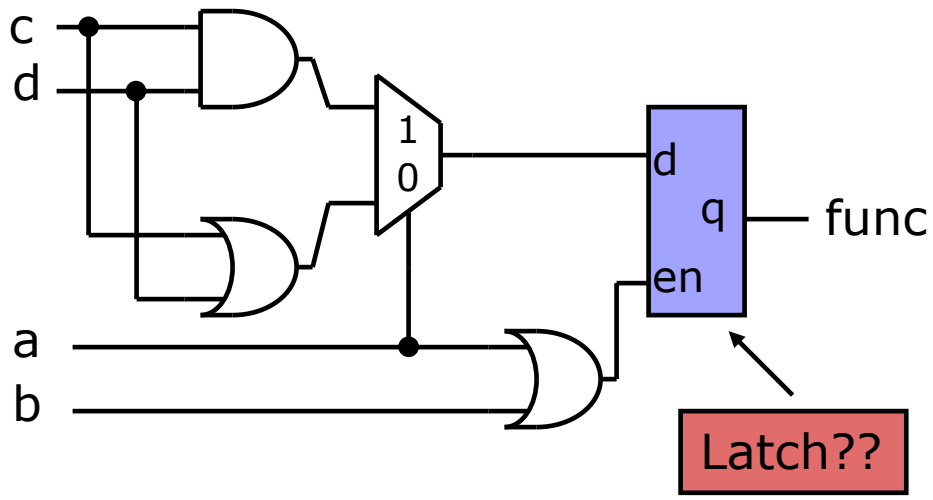


Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

if…else if…else statement

- How does this synthesize?

```
if (a)
  func = c & d;
else if (b)
  func = c | d;
```



- What you ask for is what you get!
- **func** is of type **reg/logic**
- When neither **a** nor **b** are asserted, it does not get a new value
  - That means it must have remained the value it was before
  - That implies memory… i.e., a latch!
- Always have an **else** to any **if** to avoid unintended latches when inferring **combinational** logic

Latch Avoidance in System Verilog

```
reg func;

always @(a,b,c,d)
  if (a)
    func = c & d;
  else if (b)
    func = c | d;
```

- Traditional Verilog
  - Produces a latch, but gives no warning (unless hdlin_check_no_latch is set true)
  - **func** has to be of type **reg** even though intent is not to infer a sequential element

```
logic func;

always_comb
  if (a)
    func = c & d;
  else if (b)
    func = c | d;
```

- System Verilog
  - Produces a latch, but gives warning since always block said we intended combinational.
  - No sensitivity list needed
  - **func** can be of type **logic**, which makes more sense than **reg**

Defaulting Outputs in always_comb

- A good way to avoid latches is to **default** your outputs at the beginning of an **always_comb** block

```
module compare_4bit_behave(output reg A_lt_B, A_gt_B, A_eq_B, input [3:0] A, B);

  always_comb begin
    //// default outputs to prevent latches ////
    A_lt_B = 0;
    A_gt_B = 0;
    A_eq_B = 0;
    if (A==B)
      A_eq_B = 1;
    else if (A<B)
      A_lt_B = 1;
    else
      A_gt_B = 1;
  end
endmodule
```

```
always_comb
  if (A==B) begin
    A_lt_B = 0;
    A_eq_B = 1;
    A_gt_B = 0;
  end else if (A<B) begin
    A_lt_B = 1;
    A_eq_B = 0;
    A_gt_B = 0;
  end else begin
    A_lt_B = 0;
    A_eq_B = 0;
    A_gt_B = 1;
 end
```

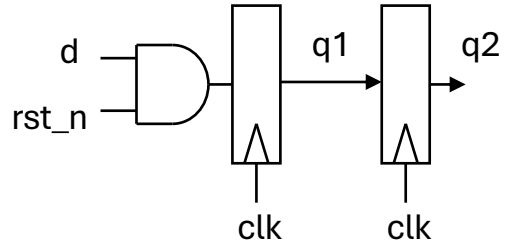
Mixing Flip-Flop Styles

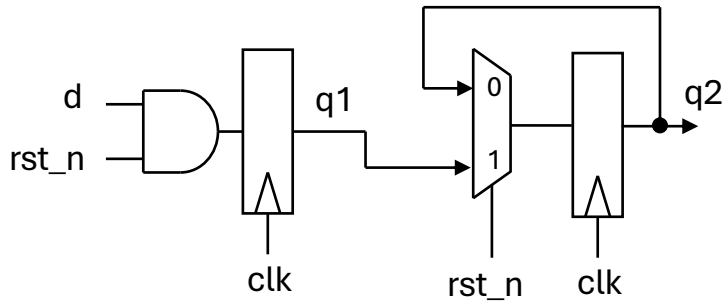
- What will this synthesize to?

```
module badFFstyle (output reg q2, input d, clk, rst_n);
  reg q1;

  always @(posedge clk)
    if (!rst_n)
      q1 <= 1'b0;
    else begin
      q1 <= d;
      q2 <= q1;
    end
endmodule
```

If !rst_n then **q2** is not assigned…
it has to keep its prior value.
Implied: **q2 <= q2;**





Mixing Flip-Flop Styles

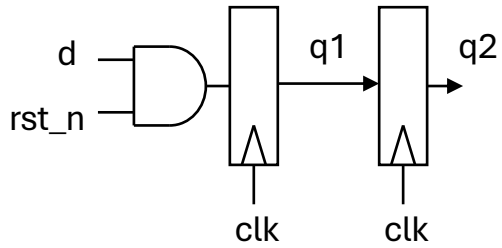
- Only combine like flops in the same always block
  - Same reset structure
  - Same enable/update condition
- Separate always blocks otherwise

```
module goodFFstyle (output reg q2, input d, clk, rst_n);
  reg q1;

  always @(posedge clk)
    if (!rst_n) q1 <= 1'b0;
    else        q1 <= d;

  always @(posedge clk)
    q2 <= q1;

endmodule
```



case statements

- System Verilog has 4 types of case statements:
  - case, casex, casez, case() inside
- General form:

```
case (expression)
   item1 : statement1;   // any of these statements could
   item2 : statement2;   // be a compound statement using
   item3 : statement3;   // begin/end
   default : statement4  // always use default for synth stuff
endcase
```

- Performs bitwise match of expression and case item
  - Both must have same bitwidth to match

- case
  - Can detect x and z
  - Good for testbenches & when no wildcards needed

case statements

- Case example:

```
localparam AND = 2'b00;
localparam OR = 2'b01;
localparam XOR = 2'b10;

always_comb
  case (alu_op)
    AND     : alu = src1 & src2;
    OR      : alu = src1 | src2;
    XOR     : alu = src1 ^ src2;
    default : alu = src1 + src2;
  endcase
```

Why always have a default?

Same reason as always having an else with an if statement.

All cases are specified, therefore no unintended latches.

```
// ALU only does AND, OR, XOR
always_comb
  case (alu_op)
    AND     : alu = src1 & src2;
    OR      : alu = src1 | src2;
default  XOR     : alu = src1 ^ src2;
  endcase
```

case statements

- Case example to detect x and z:

```
case (sig)
  1'bz:    $display("Signal is floating.");
  1'bx:    $display("Signal is unknown.");
  default: $display("Signal is %b.", sig);
endcase
```

- Use this only in a testbench!

case statements

- casex, casez
  - Use x, z, ? as wild cards (in both case expression and case item)
  - Made them error prone
  - Obsoleted by case() inside

```
always_comb begin
  casex (op_code)                    // case expression
    2'b0?: control = 8'b00100110; // case item1
    2'b10: control = 8'b11000010; // case item 2
    2'b11: control = 8'b00111101; // case item 3
    default: $display("opcode invalid");
  endcase
end
```

Uses first match encountered

- What is the output for code = 2'b01
- What is the output for code = 2'b1x

- case () inside
  - Use x, z, ? as don't care bits in case items only
  - Good for decoding instructions where some bits are don't cares

case statements

- Uses x, z, ? as wildcards in case item only
  - case () inside really shines when performing instruction decoding for most processor ISAs
  - This example shows a possible decode of a few ARM thumb opcodes

```
always_comb begin
  case (thumb_opcode) inside
    6'h001???: begin     // thumb LD/ST indirect+imm
    ...
    6'b00????: begin     // thumb immediate instructions
    ...
    6'b0101??: begin     // thumb LD/ST indirect
    ...
  endcase
end
```

- Might be good to remember this if you take 552 or 554!

Use of Range with **case () inside**

- Can use ranges in case item

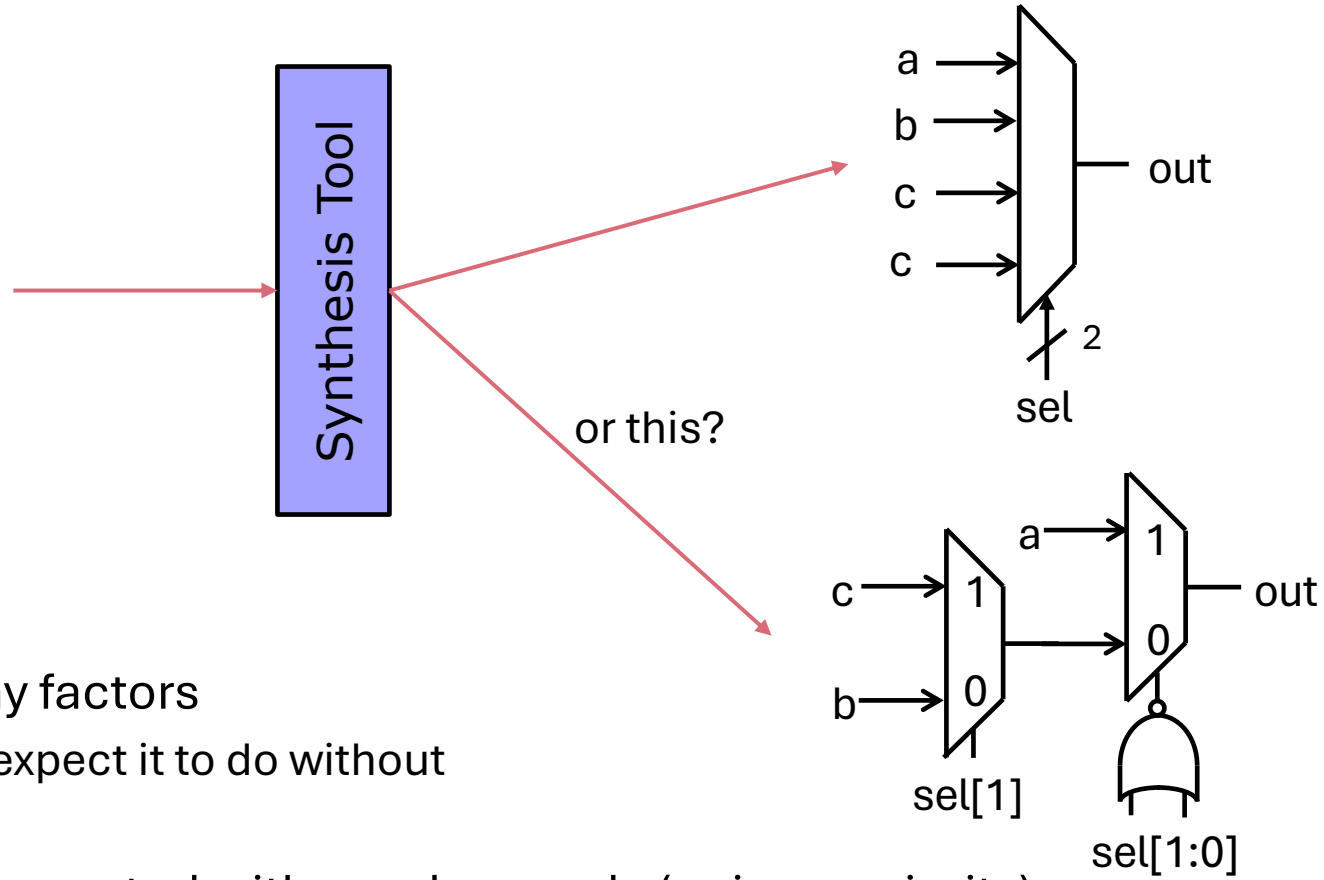
```
always_comb
  case (graduation_range) inside
    [1950:1959] : $display("Did you kiss at the drive in?");
    [1960:1969] : $display("How many Beatles records did you own?");
    [1970:1979] : $display("Are you sad disco died?");
    [1980:1989] : $display("How big was your hair back then?");
    [1990:1999] : $display("Were you hip hop or grunge?");
  endcase
```

- This range checking is synthesizable and could also be useful in decode applications.

How does a **case** statement synthesize?

- Remember the first match is taken → there is an inferred priority

```
always_comb
  case (sel)
    2'b00 : out = a;
    2'b01 : out = b;
    default : out = c;
  endcase
```

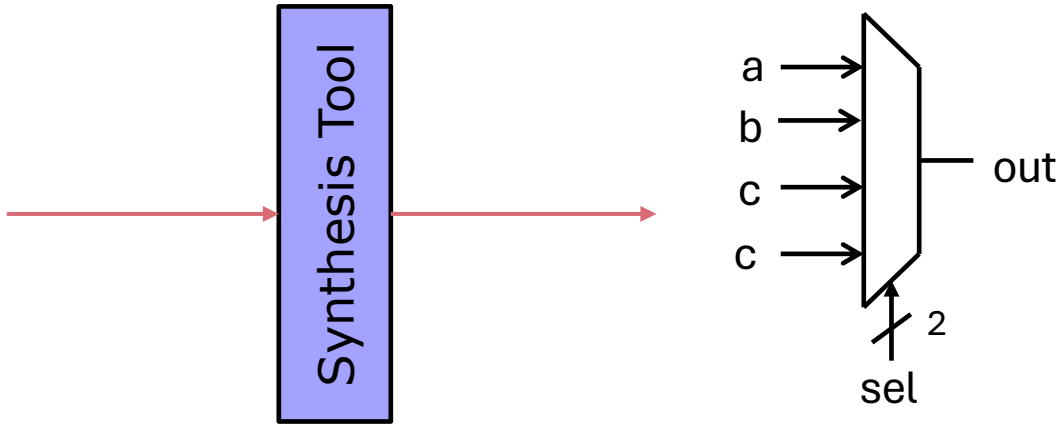


- Which one depends on many factors
  - Here, cascaded is what we expect it to do without any synthesis directives.
- System Verilog gives us more control with new keywords (unique, priority)

System Verilog **case** Options

- **unique** means one and only one case will match
  - This means it can be synthesized flat

```
always_comb
  unique case (sel)
    2'b00 : out = a;
    2'b01 : out = b;
    default : out = c;
  endcase
```




```
always_comb begin
  nxt_state = state1hot;
  unique case (1'b1)
    state1hot[0] : nxt_state = 3'b010;
    state1hot[1] : nxt_state = 3'b100;
    state1hot[2] : nxt_state = 3'b001;
  endcase
end
```

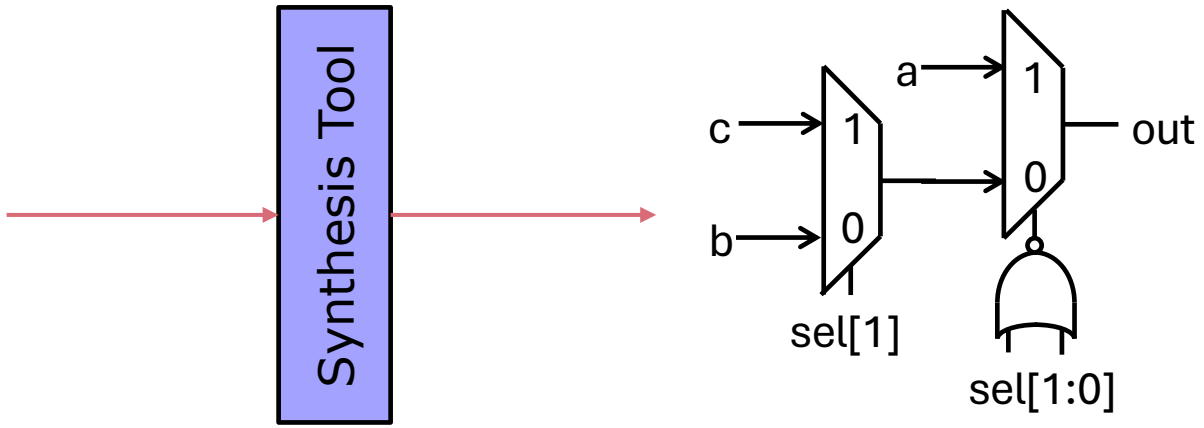
This would throw a warning in synthesis because it does not cover all cases.

This would also throw a warning in simulation if **state1hot** was 3'b111 because multiple conditions would match.

System Verilog **case** Options

- **priority** tells the synthesizer and simulator that at least one of the cases will match

```
always_comb
  priority case (sel)
    2'b00 : out = a;
    2'b01 : out = b;
    2'b10 : out = c;
  endcase
```

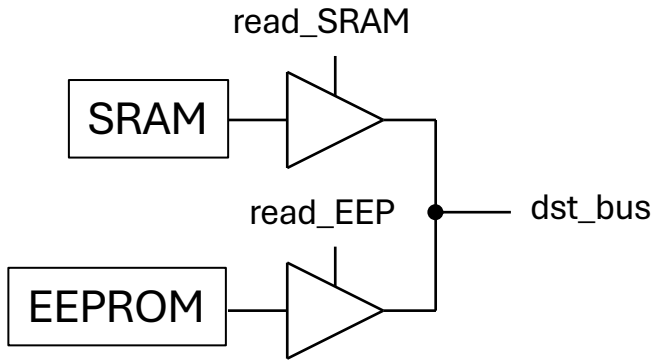


This would throw a warning in simulation if **sel** was 2'b11 because it does not match any case item.

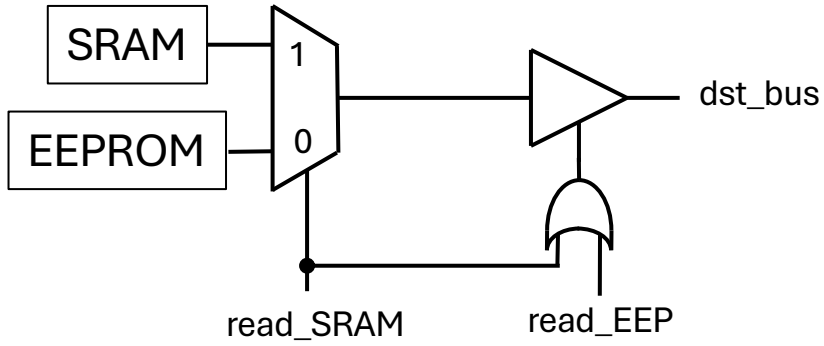
Control Constructs on if...else if...else

- The same **unique** and **priority** keywords can be applied to **if** statements

```
always_comb begin
    unique if (read_SRAM)
       dst_bus = RAM_data;
    else if (read_EEP)
       dst_bus = EEP_data;
    else
       dst_bus = 16'hzzzz;
end
```




```
always_comb begin
    priority if (read_SRAM)
       dst_bus = RAM_data;
    else if (read_EEP)
       dst_bus = EEP_data;
    else
       dst_bus = 16'hzzzz;
end
```



Encoder Synthesis

- How do you think this will synthesize?

```
module encoder (output reg [2:0] Code,
                input [7:0] Data);

  always_comb
    // encode the data
    case (Data)
      8'b00000001 : Code = 3'd0;
      8'b00000010 : Code = 3'd1;
      8'b00000100 : Code = 3'd2;
      8'b00001000 : Code = 3'd3;
      8'b00010000 : Code = 3'd4;
      8'b00100000 : Code = 3'd5;
      8'b01000000 : Code = 3'd6;
      8'b10000000 : Code = 3'd7;
      // otherwise invalid, so don't care
      default     : Code = 3'bxxx;
  endcase

endmodule
```

Will this synthesize larger or smaller?

```
module priority_encoder (output reg [2:0] Code,
                         input [7:0] Data);

  always_comb
    // encode the data
    case (Data) inside
      8'b1xxxxxxx : Code = 7;
      8'b01xxxxxx : Code = 6;
      8'b001xxxxx : Code = 5;
      8'b0001xxxx : Code = 4;
      8'b00001xxx : Code = 3;
      8'b000001xx : Code = 2;
      8'b0000001x : Code = 1;
      8'b00000001 : Code = 0;
      // should be at least one 1, don't care
      default     : Code = 3'bxxx;
  endcase
endmodule
```

Inter vs Intra Statement Delays

- Inter-assignment delays block both evaluation and assignment
  - #4 c = d;
  - #8 e = f;
- Intra-assignment delays block assignment but not evaluation
  - c = #4 d;
  - e = #8 f;
- Blocking statement is still blocking though
  - Evaluation of next statement's RHS still comes after assignment of the previous statement's LHS.

```
module inter();
integer a,b;
initial begin
  a=3;
  #6 b = a + a;
  #4 a = b + a;
end
endmodule
```

Time	Event
0	a=3
6	b=6
10	a=9

```
module intra();
integer a,b;
initial begin
  a=3;
  b = #6 a + a;
  a = #4 b + a;
end
endmodule
```

Time	Event
0	a=3
6	b=6
10	a=9

Inter vs Intra Statement Delays with Blocking

```
module inter2();
integer a,b;

initial begin
  a=3;
  #6 b = a + a;
  #4 a = b + a;
end

initial begin
  #3 a=1;
  #5 b=3;
end

endmodule
```

Time	Event
0	a=3
3	a=1
6	b=2
8	b=3
10	a=4

```
module intra2();
integer a,b;

initial begin
  a=3;
  b = #6 a + a;
  a = #4 b + a;
end

initial begin
  #3 a=1;
  #5 b=3;
end

endmodule
```

Time	Eval Event	Assign Event
0	nxt_b=6	a=3
3		a=1
6	nxt_a=7	b=6
8		b=3
10		a=7

Inter vs Intra Statement Delays with Non-Blocking

- Inter-assignment delay with non-blocking
  - Delays both the evaluation and the update
  - Effectively becomes a blocking statement

```
always @(posedge clk) begin
     b <= a + a;
  #5 c <= b + a;
  #2 d <= c + a;
end

initial begin
    a = 3; b = 2; c = 1;
end
```

Time	Event
0	clk pos edge
0	b=6
5	c=9
7	d=12

- Intra-assignment delay with non-blocking
  - Delays the update, but not the evaluation
  - Does not block

```
always @(posedge clk) begin
     b <= a + a;
     c <= #5 b + a;
     d <= #2 c + a;
end

initial begin
    a = 3; b = 2; c = 1;
end
```

Time	Event
0	clk pos edge
0	b=6
2	d=4
5	c=5

This is more like modeling the Clk2Q delay of a Flop
(it captures on rising edge, but has a delay till output)